

Project 1

Mini Sudoku

Author

Lucy Blanks

Course

CSC-17A

Section

42419

Due Date

May 24, 2026

~ Table of Contents ~

<u>1. Introduction</u>	<u>2</u>
<u>2. How to Play</u>	<u>2</u>
<u>3. Development Process</u>	<u>3</u>
<u>4. Pseudocode</u>	<u>4</u>
<u>5. Flowchart</u>	<u>5</u>
<u>6. Concepts Utilized</u>	<u>6</u>
<u>Chapter 9: Pointers/Memory Allocation</u>	<u>6</u>
<u>Chapter 10: Char Arrays and Strings</u>	<u>6</u>
<u>Chapter 11: Structured Data</u>	<u>6</u>
<u>Chapter 12: Binary Files</u>	<u>7</u>
 <u>7. References</u>	 <u>7</u>
 <u>8. Program</u>	 <u>8</u>

~ 1 Introduction ~

Most games out there are for the purpose of entertainment, connecting with others, and as a means of passing the time. Though very few offer the opportunity for significant benefits in brain cognition unlike Sudoku, one of my personal favorite puzzle games.

There are various sources that have proven that frequent challenging of the mind by playing games such as Sudoku can sharpen attention, memory, and overall thinking skills. Symptoms of dementia, for example, can be delayed by training your brain often with them. Tackling such issues become more important as people get older, but what's to say that younger people cannot work to sharpen their minds also?

Young children and adults have been exposed to short-form media in the last few years, greatly declining their attention spans at a concerning rate. Introducing younger people to Sudoku would be critical in combating the cognitive decline that mass media has produced. However, if we want to encourage them to play it actively, it wouldn't be the best idea to throw them into a full 9x9 game and expect them to understand immediately. This is why I decided to create a mini Sudoku with a 6x6 board, separated with 2x3 boxes to allow newcomers to Sudoku (and long-time players as well) to challenge themselves on a smaller scale to build their cognitive skills.

~ 2 How to Play ~

In a 6x6 board, each row, column, and individual box will be a combination of 1 to 6. Fill in the empty spaces that satisfy the numbers in the rows, columns, and the box you chose. You will choose by first entering the column, the row, and what number you believe is in that row. Depending on your answer, you will either see a message that you were correct or incorrect. Every correct answer will add your answer to the board until you fill every empty space and win!

~ 3 Development Process ~

I began with developing a quick vector generating algorithm. I needed loops that filled a 2 dimensional vector with random numbers between 1 to 6. Every random number generated was checked and made sure that it was a unique number that did not match with the second vector of the same index, and it did not match with any other elements in its own vector. I then had to develop a check for a unique number every third index twice in order to build the boxes required for a sudoku game. Once a successful 6x6 board was able to be generated, I chose 15 random indexes in both vectors to change their values to zeroes to be stored in a separate 2D vector via a board structure that contained an unsolved “puzzle” board and the originally generated “solved” board. The user would initially play the game in a loop and insert a number. If the number matched with the value in the “solved” vector of the same indexes, then the user would receive a message and the “puzzle” board would update to fill that initial 0 with the original number until the puzzle vector matched the solved vector, declaring a win. After making the basic game, I added a menu with a UI, and an option for the user to leave the sudoku game at any point between solving the puzzle. I then made files which contained the player’s name, the solved vector and the puzzle vector. The rest of the process was finding ways to utilize pointers and memory allocation since my initial code did not contain any of those concepts before.

~ Pseudocode ~

Initialize

Display Sudoku message

Call the getFile() function

Display returning player message

Prompt input for y (yes) or n (no)

If input is yes

Load file

If input is no

Enter name

Create save file

Call the menu() function

Display message to choose between options 1-5

Prompt input for a number

If input is 1

Call the genbrd() function and generate solved board

Set puzzle board equal to solved board

Add random zeroes to puzzle board

Print puzzle board

Call the play() function

Prompt menu to play or go back to menu

Inputs 1 or 2

If input is 1

Enter a row between 1-6

Enter a column between 1-6

Enter a number to fill in space

Check if number is correct or not

If input is 2

Go back to menu

If input is 2

If puzzle board has no zeros (User has already solved their board)

Generate new board

Call the play() function

If input is 3

Save the game with function save()

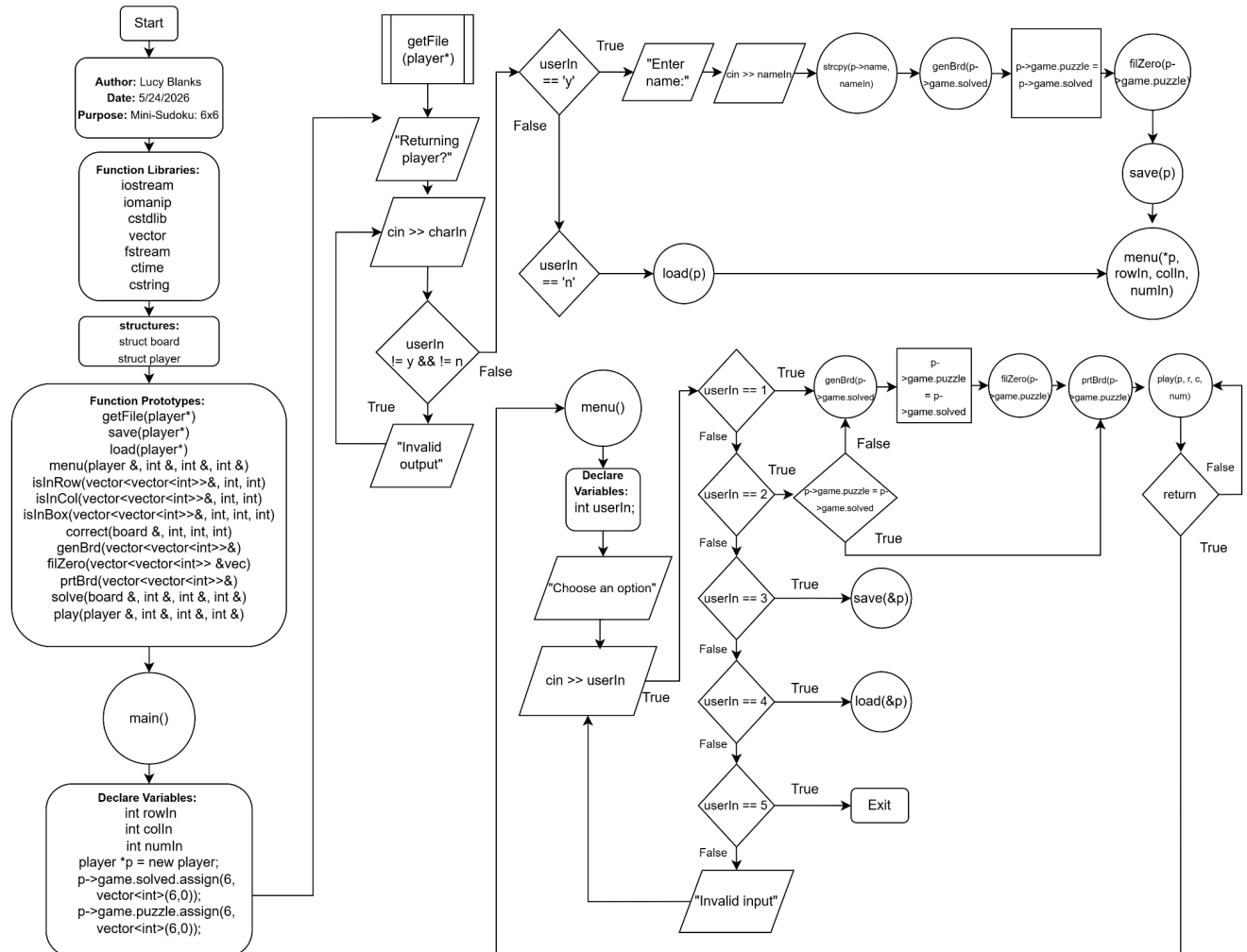
If input is 4

Load existing game with function load()

If input is 5

Quit game

~ Flowchart ~



~ Concept Utilized ~

~ Chapter 9 Pointers / Memory Allocation~

Name	Frequency	Location
Pointer Variables	1	Line 63
Arrays/Pointers	4	Various lines
Function parameters	3	Lines 38, 39, 40
Memory allocation	3	Lines 63, 76, 77
Return parameters	12	Lines 133, 160, 174, 201, 272, 276, 282, 286, 298, 303, 308, 310

~ Chapter 10 Char Arrays and Strings~

Name	Frequency	Location
C-Strings	1	Line 107

~ Chapter 11 Structured Data~

Name	Frequency	Location
Arrays	3	Line 28, 29, 34
Nested	1	Line 34
Function Arguments	7	Lines 38-41, 45, 49, 50

~ Chapter 12 Binary Files ~

Name	Frequency	Location
Formatting	2	Lines 128, 169
Member functions	10	Lines 128, 138, 143, 150, 159, 169, 179, 184, 191, 200
Binary files	2	Lines 128, 169
Records with Structures	6	Lines 138, 143, 150, 179, 184, 191

~ References ~

1. Professor Lehr's Lectures and Labs
2. "Starting Out with C++: From Control Structures through Objects" Gaddis, Tony. 9th Edition (Textbook)

~ Program ~

/*

Author: Lucy Blanks

Date:5/24/2026

Purpose: Mini-Sudoku: 6x6

//v.6 ~ Additional conditions

switch player.name from string to char - DONE!

switch i/o files to binary - DONE!

declare a local pointer variable that points to board - DONE!

utilize pointers - DONE!

utilize memory allocation - DONE!

utilize delete operator - DONE!

Game is now fully functional! Enjoy!

*/

//system libraries

#include <iostream> //input/output library

#include <iomanip> //formatting library

#include <cstdlib> //random library

#include <vector> //vector library

#include <fstream> //file library

#include <ctime> //time library

#include <cstring> //string library

using namespace std; //using namespace standard

//structures

struct board {

vector<vector<int>> solved;

vector<vector<int>> puzzle;

};

struct player {

char name[50];

struct board game;

};

//function prototypes

void getFile(player*);

```

bool save(player*);
bool load(player*);
void menu(player &, int &, int &, int &);
bool isInRow(vector<vector<int>>&, int, int);
bool isInCol(vector<vector<int>>&, int, int);
bool isInBox(vector<vector<int>>&, int, int, int);
bool correct(board &, int, int, int);
void genBrd(vector<vector<int>>&);
void filZero(vector<vector<int>> &vec);
void prtBrd(vector<vector<int>>&);
void solve(board &, int &, int &, int &);
void play(player &, int &, int &, int &);

```

```

//execution begins here at main
int main(int argc, char **argv ) {
    //set random number seed
    srand(time(0));

    //declare variables
    int rowIn;
    int colIn;
    int numIn;

    //initialize variables
    player *p = new player;
    p->game.solved.assign(6, vector<int>(6,0));
    p->game.puzzle.assign(6, vector<int>(6,0));

    //introduce program
    cout << endl;
    cout << "MINI SUDOKU" << endl;

    //load or create new file
    getFile(p);
    menu(*p, rowIn, colIn, numIn);

    //clean up memory, close files
    delete p;
    p = nullptr;

    //exit the program
    return 0;
}

```

```

//functions
void getFile (struct player *p) {
    //declare variables
    char charIn;
    char nameIn[50];

    //determine if user is returning or new
    do {
        cout << "Is this your first time playing?" << endl;
        cout << "y = yes\nn = no" << endl;
        cin >> charIn;

        //validate input
        if (charIn != 'y' && charIn != 'Y' && charIn != 'n' && charIn != 'N') {
            cout << "Invalid input." << endl;
            cout << endl;
        }
    } while (charIn != 'y' && charIn != 'Y' && charIn != 'n' && charIn != 'N');

    //if user is new
    if (charIn == 'y' || charIn == 'Y') {
        //collect name from user
        cout << "Please enter your name: " << endl;
        cin >> nameIn;
        strcpy(p->name, nameIn);

        //generate new board
        genBrd(p->game.solved);
        p->game.puzzle = p->game.solved;
        fillZero(p->game.puzzle);
        save(p);
    }

    //if user is returning, load data
    if (charIn == 'n' || charIn == 'N') {
        load(p);
    }
}

bool save (struct player *p) {
    //declare variables
    fstream file;

```

```

//open file
file.open ("save.txt", ios::out | ios::binary);

//validate file
if (!file) {
    cout << "File not found" << endl;
    return false;
}
else {
    //write values to .txt file
    //player name
    file.write(reinterpret_cast<char *>(&p->name),sizeof(p->name));

    //puzzle
    for (int i = 0; i < 6; i++) {
        for (int j = 0; j < 6; j++) {
            file.write(reinterpret_cast<char *>(&p->game.puzzle[i][j]),sizeof(int));
        }
    }

    //solved
    for (int i = 0; i < 6; i++) {
        for (int j = 0; j < 6; j++) {
            file.write(reinterpret_cast<char *>(&p->game.solved[i][j]),sizeof(int));
        }
    }

    //confirm save
    cout << endl;
    cout << "Saved to 'save.txt'" << endl;

    //close file
    file.close();
    return true;
}
}

bool load (struct player *p) {
    //declare variables
    fstream file;

    //open file
    file.open ("save.txt", ios::in | ios::binary);

```

```

//validate file
if (!file) {
    cout << "File not found" << endl;
    return false;
}
else {
    //read values to .txt file
    //player name
    file.read(reinterpret_cast<char *>(&p->name),sizeof(p->name));

    //puzzle
    for (int i = 0; i < 6; i++) {
        for (int j = 0; j < 6; j++) {
            file.read(reinterpret_cast<char *>(&p->game.puzzle[i][j]),sizeof(int));
        }
    }

    //solved
    for (int i = 0; i < 6; i++) {
        for (int j = 0; j < 6; j++) {
            file.read(reinterpret_cast<char *>(&p->game.solved[i][j]),sizeof(int));
        }
    }

    //confirm load
    cout << endl;
    cout << "Welcome back, " << p->name << "." << endl;

    //close file
    file.close();
    return true;
}
}

void menu(struct player &p, int &r, int &c, int &num){
    //declare variables
    int userIn;

    do {
        //display menu
        cout << endl;
        cout << "Player: " << p.name << endl;
        cout << "Choose an option: " << endl;
        cout << "1. Play New Game" << endl;
    } while (userIn != 1);
}

```

```

cout << "2. Continue Game" << endl;
cout << "3. Save" << endl;
cout << "4. Load" << endl;
cout << "5. Exit" << endl;
cin >> userIn;

//menu options based on input
switch (userIn) {
    //new game
    case 1 : //generate new board
        genBrd(p.game.solved);

        //match boards
        p.game.puzzle = p.game.solved;

        //create and print puzzle based on complete board
        filZero(p.game.puzzle);
        prtBrd(p.game.puzzle);
        play (p, r, c, num);
        break;

    //continuting game
    case 2 : //generate new board if there is only a blank board saved
        if (p.game.puzzle == p.game.solved) {
            cout << "You already won! Generating new board..." << endl;
            genBrd (p.game.solved);
            p.game.puzzle = p.game.solved;
            filZero (p.game.puzzle);
        }

        //play game as normal
        prtBrd (p.game.puzzle);
        play (p,r,c,num);
        break;

    case 3 : //save game
        save(&p);
        break;

    case 4 : //load game
        load(&p);
        break;

    case 5 : //quit game

```

```

        return;
        break;

    default : cout << endl;
              cout << "Invalid input." << endl;
    }
}while (userIn != 5);
}

```

```

bool islnRow(vector<vector<int>> &vec, int row, int n) {
    for (int i = 0; i < 6; i++) {
        if (vec[row][i] == n) {
            return true;
        }
    }

    return false;
}

```

```

bool islnCol(vector<vector<int>> &vec, int col, int n) {
    for (int i = 0; i < 6; i++) {
        if (vec[i][col] == n) {
            return true;
        }
    }

    return false;
}

```

```

bool islnBox(vector<vector<int>> &vec, int row, int col, int n) {
    //declare variables
    int rBox = row - (row % 2);
    int cBox = col - (col % 3);

    //verify 2x3 box
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            if (vec[rBox + i][cBox + j] == n) {
                return true;
            }
        }
    }
}

```

```

    return false;
}

bool correct (board &b, int r, int c, int n) {
    if (b.solved[r][c] == n) {
        return true;
    } else {
        return false;
    }
}

void genBrd(vector<vector<int>> &b) {
    //declare variables
    int digit;

    //fill vector, validate rows are unique numbers
    for (int i = 0; i < 6; i++){
        for (int j = 0; j < 6; j++) {
            //declare variables
            bool failed = false;
            int tries = 0;

            do {
                //generate random number
                digit = rand() % 6 + 1;
                tries++;

                //backtrack
                if (tries > 1000) {
                    for (int k = 0; k < 6; k++) {
                        b[i][k] = 0;
                    }
                    j = -1;
                    tries = 0;
                    failed = true;
                    break;
                }
            } while (isInRow(b, i, digit) || isInCol(b, j, digit) || isInBox (b, i, j, digit));

            if (!failed) {
                b[i][j] = digit;
            }
        }
    }
}

```



```

    }
}

void filZero(vector<vector<int>> &vec) {
    //declare variables
    int count = 0;

    while (count < 15) {
        //geneate random row and column to pick from
        int r = rand() % 6;
        int c = rand() % 6;

        //fill chosen element to be zero
        if (vec[r][c] != 0) {
            vec[r][c] = 0;
            count++;
        }
    }
}

```

```

void prtBrd(vector<vector<int>> &b){
    //top of board
    cout << "  1  2  3  4  5  6" << endl;
    cout << "-----" << endl;

    for (int i = 0; i < 6; i++) {

        //horizontal divider
        if (i % 2 == 0 && i > 0) {
            cout << "|-----|-----|" << endl;
        }

        for (int j = 0; j < 6; j++) {

            //vertical divider
            if (j % 3 == 0) {
                cout << "|";
            }

            //display sign if there is an empty space
            if (b[i][j] == 0) {
                cout << " - ";
            }
        }
    }
}

```

```

        } else {
            cout << " " << b[i][j] << " ";
        }

    }
    //separate rows and skip lines
    cout << "|" << endl;
}

//bottom of board
cout << "-----" << endl;
}

void solve (board &b, int &r, int &c, int &n) {
    //collect values from user
    //collect rows
    do {
        cout << "Enter row (horizontal):" << endl;
        cin >> r;
    } while (r > 6 || r <= 0);

    //collect columns
    do {
        cout << "Enter column (vertical):" << endl;
        cin >> c;
    } while (c > 6 || c <= 0);

    //verify numbers
    if (b.puzzle[r - 1][c - 1] != 0) {
        cout << endl;
        cout << "There is already a number there!" << endl;
        cout << endl;
    } else {
        cout << "Enter a number:" << endl;
        cin >> n;

        if (correct(b, r - 1, c - 1, n) == true) {
            cout << endl;
            cout << "You got it!" << endl;
            cout << endl;
            b.puzzle[r - 1][c - 1] = n;
        } else {
            cout << endl;
            cout << "Incorrect!" << endl;
        }
    }
}

```

```

        cout << endl;
    }
}

}

void play (player &p, int &r, int &c, int &n) {
    //declare variables
    int userIn;
    board *boardPtr = &p.game;

    do {
        //display menu
        cout << "Enter an option: " << endl;
        cout << "1. Play \n2. Quit" << endl;
        cin >> userIn;

        switch (userIn) {
            case 1 : solve(*boardPtr, r, c, n);
                    prtBrd(boardPtr->puzzle);
                    break;
            case 2 : return;
            default : cout << endl;
                     cout << "Invalid input." << endl;
        }
    } while (boardPtr->puzzle != boardPtr->solved);

    //save win to file
    cout << "You win!!" << endl;
    save(&p);
}

```